Ottimizzazione di Strategie di Software Effort Estimation

Analisi comparativa e validazione sperimentale del Modello Fuzzy-DCNN

CORSO DI SOFTWARE PROJECT MANAGEMENT AND EVOLUTION

Fabrizio Apuzzo **Agostino Cesarano**

21 agosto 2026

Abstract

Il presente documento fornisce un'analisi approfondita dello stato dell'arte nella stima dello sforzo di sviluppo software (*Software Development Effort Estimation*, SDEE), esplorando, confrontando ed ottimizzando due approcci di rilievo scientifico recente.
L'articolo di Di Martino et al. [3] valuta empiricamente l'efficacia di cinque metodi approssimati di misurazione della dimensione funzionale (*Functional Size Measurement*, FSM) — basati su FPA e COSMIC — forniti in input a modelli di Regressione Lineare Semplice (SLR).
L'articolo di Azzeh et al. [2] propone invece un modello di deep learning denominato **F-DCNN**, che combina una fase di clusterizzazione fuzzy (*Fuzzy C-Means*, FCM) con una rete neurale convoluzionale profonda (DCNN) bidimensionale.
Nel presente lavoro viene presentata una **campagna di ottimizzazione del modello F-DCNN** condotta su 5 dataset benchmark internazionali (*COCOMO-81*, *Albrecht*, *Desharnais*, *Maxwell*, *China*). Vengono affrontati ed abbattuti i limiti empirici della baseline originale attraverso cinque assi di intervento: (1) validazione geometrica della scelta del numero di cluster fuzzy tramite Silhouette Score, Davies-Bouldin e Calinski-Harabasz Index; (2) mitigazione completa dei neuroni morti (*dying ReLU*) tramite layer di output lineare con clipping in inferenza; (3) ottimizzazione iperparametrica a due stadi; (4) introduzione della Huber Loss ($\delta = 0.01$) per gestire la presenza di outlier; (5) stabilizzazione della varianza mediante trasformazione logaritmica del target ($\log(1 + y)$). I risultati dimostrano miglioramenti drastici nell'errore mediano (MedAE), riducendo l'errore su *China* da 690.07 a 277.91 ($p = 2.41 \times 10^{-22}$ con test di Wilcoxon). Viene infine fornita un'analisi dettagliata del bilanciamento tra complessità e benefici predittivi ed è stato realizzato un prototipo applicativo web interattivo per la stima dell effort in tempo reale.

1 Introduzione

La stima dei costi e dell' effort di realizzazione di progetti software è un'attività critica per la pianificazione, la gestione del budget e il successo industriale del software. Tuttavia, la

natura intrinsecamente complessa, dinamica ed incerta del processo di ingegnerizzazione rende questa attività particolarmente ardua. Negli ultimi decenni la letteratura scientifica ha proposto una molteplicità di modelli, spaziando dai tradizionali approcci algoritmici parametrici (es. COCOMO) alle tecniche di intelligenza artificiale ed apprendimento automatico.

Un aspetto fondamentale emerso dalla ricerca è che la qualità delle stime non dipende unicamente dalla sofisticazione del predittore matematico, ma anche dalla forma e dalla rappresentazione dei dati di input forniti al modello. Da un lato, approcci come quello di Di Martino et al. [3] mantengono fisso il predittore (utilizzando modelli di regressione lineare semplice) e valutano l'impatto di diverse approssimazioni delle misure funzionali (FPA e COSMIC) per ridurre i costi di misurazione manuale. Dall'altro lato, approcci di deep learning ibrido come F-DCNN di Azzeh et al. [2] trasformano lo spazio delle feature tabulari vettoriali in matrici di appartenenza fuzzy bidimensionali per permettere l'applicazione di filtri convoluzionali 2D che permettono un'estrazione di caratteristiche più efficace.

> **Obiettivo del Lavoro:** Colmare le lacune metodologiche del modello F-DCNN baseline (instabilità dell'indice Xie-Beni, neuroni morti in output, elevata varianza), fornendo sia un quadro teorico aggiornato dello stato dell'arte, sia una formulazione ottimizzata validata statisticamente e fruibile mediante un'applicazione web prototipale.

2 Lo stato dell'arte nella stima dello sforzo software

La letteratura scientifica in ambito Software Engineering [9, 6] individua e categorizza quattro principali macro-approcci per la stima dello sforzo: stime basate su algoritmi, stime basate sul giudizio di esperti, stime basate su modelli di machine learning, e metodi ibridi. Ad essi si aggiunge il recente filone basato su Large Language Models.

2.1 Stime basate su algoritmi

I metodi algoritmici applicano formule matematiche e modelli statistici empirici dedotti da dati storici di progetti pregressi. È essenziale tracciare una distinzione concettuale rigorosa tra due sotto-famiglie di metodi aventi profili di costo ed economicità nettamente differenti:

- **Metodi di stima dello sforzo (Effort Estimation Methods):** modelli come COCOMO (Constructive Cost Model) e SLIM, che calcolano direttamente lo sforzo di sviluppo E (espresso tipicamente in mesi-uomo o ore-persona) a partire da una misura della dimensione del software S già disponibile, combinandola con opportuni *cost driver* legati all'ambiente ed al team di sviluppo:
$$E = a \cdot S^b \cdot \prod_i EM_i \tag{1}$$
Una volta noti i valori dei *cost driver*, tali modelli applicano una formula in modo rapido ed economico, senza richiedere necessariamente l'intervento continuo di un esperto nella fase di calcolo finale [4].
- **Metodi di stima della dimensione (Functional Size Measurement, FSM):** metodologie standardizzate ISO come Function Point Analysis (FPA IFPUG) e COSMIC, il cui scopo non è calcolare lo sforzo, ma determinare una misura oggettiva della dimensione funzionale espressa in Function Point (FP) o COSMIC Function Points (CFP) da fornire come dato di input ai metodi della prima categoria.

I metodi di misurazione della dimensione (FSM) richiedono l'intervento di misuratori esperti e certificati: la caratterizzazione delle componenti funzionali del sistema (distinzione tra File

Logici Interni, Interfacce Esterne, Input, Output ed Inchieste, unitamente alla determinazione della loro complessità in termini di DET e RET) è un compito complesso e costoso. Gli studi sulla produttività attestano ritmi di misurazione lenti, compresi tra **200 e 600 FP al giorno** [3], fattore che ha fortemente motivato lo sviluppo di approcci di approssimazione veloci.

Inoltre, l'efficacia di entrambe le categorie di modelli algoritmici è limitata dalla rigidità nel declinarsi in contesti tecnologici e metodologici differenti. La letteratura [7] evidenzia consistenti criticità nell'impiego del modello COCOMO II a seguito della capillare diffusione di paradigmi Agile, microservizi e servizi Cloud, rendendo necessaria una costante ricalibrazione dei parametri o la proposta di evoluzioni quali COCOMO III.

2.2 Stime basate sul giudizio di esperti

Tali approcci si fondano sull'esperienza accumulata dai professionisti del settore per valutare i costi ed i tempi di realizzazione del software. Sebbene soggetti a soggettività e bias cognitivi, la letteratura evidenzia come l'esperienza diretta permetta di catturare sfumature di contesto non formalizzabili numericamente; lo studio sistematico di Jørgensen e Shepperd [6] evidenzia infatti che in **10 dei 16 studi empirici analizzati** le stime basate sul giudizio di esperti hanno registrato un vantaggio in termini di accuratezza rispetto ai metodi automatici o algoritmici.

Tra le metodologie più diffuse figurano:

- **Planning Poker e Delphi Wideband:** processi iterativi e anonimi in cui più esperti stimano in parallelo lo sforzo per completare un'attività, confrontando le stime e discutendo le discrepanze fino al raggiungimento del consenso.
- **Analisi degli Story Point:** misurazione relativa della complessità e dello sforzo percepito in ambiti Agile, in cui lo sforzo viene espresso rispetto a baseline di attività note.
- **Insiemi Fuzzy ed Expert System:** modellazione della vaghezza dell'informazione tramite funzioni di appartenenza definite da esperti domain-specific per rappresentare l'incertezza su ogni componente del progetto.

2.3 Stime basate su modelli di apprendimento automatico

I modelli di Machine Learning (ML) apprendono direttamente dai dati storici relazioni complesse ed altamente non lineari tra le caratteristiche dei progetti e lo sforzo finale. Tali modelli offrono un'elevata capacità di adattamento a contesti eterogenei e possono essere continuamente aggiornati con i dati dei progetti appena conclusi. Tuttavia, richiedono dataset ben bilanciati e presentano costi operazionali legati alla complessità ed all'interpretabilità dei risultati (*black-box effect*).

Gli approcci di machine learning maggiormente studiati nella letteratura SDEE includono:

- **Regressione Lineare Semplice e Multipla (SLR/MLR):** modelli statistici che stabiliscono relazioni matematiche lineari tra le dimensioni del software e l'effort.
- **Alberi Decisionali e Random Forest:** strutture gerarchiche di decisione singole o combinate in ensemble per incrementare la stabilità delle predizioni.
- **Support Vector Machines (SVM / SVR):** predittori basati sull'individuazione di iperpiani ottimali di separazione e regressione in spazi ad alta dimensione.
- **Gradient Boosting (XGBoost, LightGBM):** modelli ensemble sequenziali in cui ciascun albero correggere gli errori residui commessi dagli alberi precedenti.
- **Reti Neurali Artificiali e Multi-Layer Perceptron (MLP):** architetture a nodi interconnessi capaci di apprendere relazioni fortemente non lineari.

2.4 Approcci ibridi e classificazione dei modelli

I metodi ibridi combinano due o più tecniche per sfruttarne i rispettivi punti di forza (es. un modello algoritmico per la stima preliminare raffinato poi tramite esperti o intelligenza artificiale). La letteratura distingue due principali categorie:

- **Omogenei:** combinazione dello stesso modello di base configurato con parametri differenti (es. ensemble di più alberi decisionali).
- **Eterogenei:** combinazione di modelli concettualmente differenti (es. sistemi neuro-fuzzy come SEER-SEM [4] o l'unione di clustering ed algoritmi di ottimizzazione).

È essenziale chiarire che le macro-categorie della letteratura servono a fornire un quadro sistematico ma non costituiscono definizioni rigide. Nello specifico, il modello di **Di Martino et al.** non costituisce un approccio ibrido, in quanto la prima fase di preprocessing dei dati (misurazione approssimata della dimensione) non rappresenta un modello di stima dello sforzo, ma un metodo per ottenere una misura di input fornita ad un predittore di Regressione Lineare Semplice (SLR). Analogamente, il modello **F-DCNN di Azzeh et al.** descritto nella Sezione 4, pur articolandosi in due fasi distinte (clusterizzazione fuzzy e rete neurale convoluzionale profonda), rappresenta un metodo di apprendimento automatico guidato da feature engineering fuzzy e non un approccio ibrido eterogeneo propriamente detto.

2.5 Modelli basati su Transformer e Large Language Models (LLM)

Negli ultimi anni (2024–2026), l'evoluzione del Deep Learning basato sull'architettura Transformer [10] ha aperto un nuovo paradigma nella stima dello sforzo software [11, 1]. A differenza dei modelli tradizionali basati esclusivamente su metriche vettoriali o dimensionali preraccolte, i modelli basati su LLM e linguaggi di embedding semantico permettono di elaborare direttamente i testi in linguaggio naturale che descrivono i requisiti di sistema (User Stories, Software Requirement Specifications - SRS, o ticket Jira).

In particolare, la letteratura recente individua due principali modalità applicative:

1. **Misurazione Funzionale Automatizzata:** Impiego di modelli di classificazione (es. BERT, RoBERTa o DeepSeek-Coder) per identificare e categorizzare automaticamente i processi funzionali (Input Esterni, Output Esterni, File Logici Interni) direttamente dai testi delle User Stories, stimando di conseguenza i Function Point FPA o COSMIC senza l'intervento di un certificatore umano [11].
2. **Stima Diretta dello Sforzo (End-to-End Few-Shot Estimation):** Utilizzo di architetture di grandi dimensioni (LLM da 7B a 70B parametri) sollecitate tramite tecniche di *in-context learning* e *few-shot prompting*. Il modello analizza il contesto del testo dei requisiti e predice direttamente lo sforzo in ore-persona attingendo alla conoscenza memorizzata durante il pre-addestramento [1].

Sebbene gli approcci LLM offrano il notevole vantaggio di operare direttamente su documentazione non strutturata eliminando la fase manuale di estrazione delle feature, essi presentano consistenti limitazioni operative ed industriali:

- **Elevato Footprint Computazionale:** Richiedono infrastrutture di calcolo dedicate con ingenti quantità di VRAM GPU sia in addestramento che in inferenza.
- **Latenza e Dipendenza da API Esterne:** L'inferenza mediante servizi cloud comporta latenze nell'ordine di secondi, costi di chiamata ricorrenti e severe problematiche di privacy e riservatezza dei dati aziendali.
- **Non-determinismo e Allucinazioni:** Le stime basate su LLM soffrono di varianza stocastica nella generazione del testo e potenziale allucinazione numerica.

Al contrario, modelli di apprendimento automatico focalizzati su feature tabulari come la rete **Fuzzy-DCNN** proposta in questo lavoro rappresentano l'estremo opposto dello spettro tecnologico: elaborando caratteristiche numeriche sintetizzate con un'inferenza deterministica inferiore ai **5 millisecondi** su hardware CPU standard, costituiscono la soluzione ideale per l'integrazione *on-the-fly* in dashboard di supporto decisionale locale.

3 Stima dello sforzo tramite misurazioni funzionali approssimate: Di Martino et al. (2020)

L'articolo di Di Martino et al. [3] indaga empiricamente l'efficacia dell'utilizzo di misure di dimensione funzionale approssimate come input per predittori lineari dello sforzo, al fine di ridurre i tempi ed i costi del processo di sizing.

3.1 Metodo

Gli autori hanno analizzato 25 progetti software industriali sviluppati in ambito web da una singola azienda italiana. Ciascun progetto è stato misurato tramite:

- Standard FPA (IFPUG) e due sue approssimazioni: *High Level FPA* (HLFPA) e *Indicative FPA* (IFPA).
- Standard COSMIC e tre sue approssimazioni: *Average Functional Process* (AFP), *Fixed Size Classification* (FSC), ed *Equal Size Bands* (ESB).

Per ogni misura ottenuta, è stato addestrato un modello di Regressione Lineare Semplice (SLR) per stimare lo sforzo espresso in ore-persona:

$$E = \beta_0 + \beta_1 \cdot \text{Size}_{\text{approssimata}} \tag{2}$$

Gli autori hanno deliberatamente evitato l'impiego di predittori non lineari complessi o algoritmi di machine learning avanzati, poiché l'obiettivo primario dello studio era isolare e confrontare direttamente l'efficacia delle diverse misure di dimensione funzionale, mantenendo neutra ed invariante la tecnica di stima. La validazione è stata condotta tramite Leave-One-Out Cross-Validation (LOOCV), valutando la metrica Mean of Absolute Residuals (MAR) e verificando la significatività statistica mediante test di Wilcoxon ed effect size di Vargha-Delaney rispetto a due baseline costanti (media e mediana dello sforzo).

3.2 Risultati

Per le approssimazioni FPA, il modello basato su **High Level FPA** (MAR = 561 ore-persona) ha superato nettamente le baseline (MAR rispettivamente 843 e 872 ore-persona) ottenendo un'accuratezza comparabile a quella del modello FPA standard (MAR = 580), senza differenze statisticamente significative. Al contrario, **Indicative FPA** (MAR = 760) non ha mostrato differenze statisticamente significative rispetto alle baseline, risultando ampiamente sconsigliato dagli autori anche per stime di massima ordine di grandezza.

Per le approssimazioni COSMIC, tutte le varianti hanno superato significativamente le baseline con un *effect size* ampio. Confrontati con il modello COSMIC standard (MAR = 276 ore-persona), i modelli approssimati mostrano un errore via via minore al crescere della sofisticazione del metodo: **Average Functional Process** produce un errore circa il 40% superiore, **Fixed Size Classification** circa il 22% superiore, mentre **Equal Size Bands** (MAR = 295) mostra una differenza di solo il 7%, non statisticamente significativa rispetto al metodo standard.

3.3 Discussione

Gli autori concludono che le approssimazioni non sono equivalenti tra loro: **High Level FPA** ed **Equal Size Bands** rappresentano soluzioni ideali nelle fasi iniziali di progetto quando le informazioni o il tempo sono limitati, riducendo drasticamente lo sforzo di misurazione senza un degrado significativo della precisione. Al contrario, Indicative FPA, Average Functional Process e Fixed Size Classification introducono un degrado rilevante dell'accuratezza o richiedono comunque calibrazioni su progetti pregressi ed una classificazione soggettiva dei processi funzionali (con un tasso di errata classificazione in letteratura superiore al 10%).

4 Stima dello sforzo con rete convoluzionale e Clustering Fuzzy: Azzeh et al. (2024)

L'articolo di Azzeh et al. [2] propone la rete **F-DCNN** (Fuzzy Deep Convolutional Neural Network), introducendo l'uso delle CNN 2D nel dominio SDEE. A differenza dell'approccio di Di Martino et al., che valuta diverse misure di dimensione mantenendo fissa la tecnica di stima (SLR), questo lavoro sviluppa una nuova architettura di machine learning mantenendo come input le feature vettoriali tabulari tradizionali.

4.1 Motivazione e metodo

Le reti convoluzionali 2D, sebbene consolidate in ambito Computer Vision, non erano mai state applicate alla stima dello sforzo software a causa dell'incongruenza della forma dei dati (*input mismatch*):

> «the DCNN was not used before for the problem of effort estimation because of the input mismatch. The DCNN requires input in the form of 2D matrix, which is not available on the current datasets» [2].

Per superare tale limite, gli autori introducono una fase di feature engineering basata su **Fuzzy C-Means (FCM)**:

1. Il dataset vettoriale viene partizionato in k cluster fuzzy (con k selezionato mediante l'indice Xie-Beni).
2. Per ogni osservazione x_i e per ogni feature j, viene calcolato il grado di appartenenza fuzzy $\mu_{ij} \in [0, 1]$ rispetto al centroide del cluster tramite funzioni di appartenenza gaussiane.
3. Ne deriva una matrice 2D di dimensioni $(\text{feature} \times k)$ fornita in input alla rete DCNN.

L'intuizione fondante risiede nel fatto che la rappresentazione fuzzy permetta di descrivere le caratteristiche di un progetto software in modo flessibile ed articolato: un progetto può collocarsi prevalentemente all'interno di un cluster (ad esempio caratterizzato da un elevato numero di interfacce esterne e pochi file logici), ma mantenere al contempo gradi di appartenenza non nulli verso altri cluster. La rete convoluzionale apprende filtri 2D capaci di correlare tali pattern sfumati con lo sforzo di sviluppo finale.

L'architettura DCNN prevede due strati convoluzionali 2D con 16 filtri 3×3, strati di Max Pooling 2×2, ed uno strato Dense completamente connesso (32 e 16 neuroni) con attivazione finale ReLU, addestrata tramite ottimizzatore Adam e MSE Loss.

4.2 Risultati empirici del paper originale

La valutazione condotta da Azzeh et al. [2] su dataset pubblici sotto LOOCV ha mostrato prestazioni superiori rispetto a predittori tradizionali quali Analogy-Based Effort Estimation

(ABE), Support Vector Regression (SVR), Random Forest (RF), XGBoost e Multi-Layer Perceptron (MLP), ottenendo valori di Standardized Accuracy (SA) elevati e riduzioni del Mean Absolute Error (MAE).

4.3 Analisi di riproducibilità e reverse engineering del modello baseline

In fase di replica e validazione indipendente del modello F-DCNN, è emerso un consistente divario metodologico tra le metriche teoriche dichiarate da Azzeh et al. [2] e quelle effettivamente riproducibili su codice standard (Tabella 1). Un'analisi rigorosa ha permesso di evidenziare quattro aspetti cruciali:

Tabella 1: Confronto diagnostico iniziale tra la replica del baseline (Fase 1) ed i valori pubblicati da Azzeh et al. (2024).

Dataset	MAE Replica	MAE Paper	Ratio (Replica / Paper)	MedAE Replica
China	1519.79	899.00	1.69×	**688.19**
Desharnais	2277.98	1044.00	2.18×	1752.77
Maxwell	6438.75	1749.00	3.68×	3348.03
COCOMO-81	755.18	160.00	4.72×	**111.58**
Albrecht	8.65	1.10	7.86×	5.10

Dall'analisi sono emerse quattro evidenze metodologiche determinanti:

- **Bonifica architetturale e Preprocessing:** È stata rimossa una convoluzione pointwise 1×1 con Batch Normalization indebitamente presente nelle implementazioni di riferimento, ripristinando la fedele topologia convoluzionale 2D con Max Pooling. Inoltre, è stato corretto l'encoding categorico di COCOMO-81 (`dev_mode`), preservando integralmente le 17 feature descritte in letteratura.
- **Analisi delle formule di Fuzzificazione:** Sono stati individuati refusi tipografici nelle equazioni di appartenenza fuzzy del paper originale (fattore $4\delta^2$ a denominatore ed incongruenze su δ_{rj}). La nostra sperimentazione ha confermato che l'adozione della deviazione standard RMS corretta garantisce una copertura dello spazio latente ottimale senza distorsioni aritmetiche.
- **Instabilità del gradiente e Collasso Dead ReLU:** L'addestramento su target grezzi non scalati ad elevata varianza produceva gradienti superiori a 10.000, causando nel 67% dei fold di COCOMO-81 il collasso permanente dell'output a 0.0.
- **Esperimento di controllo con Random Forest e Ipotesi MedAE:** Poiché il modello Random Forest è invariante per trasformazioni monotone delle feature e dello scaling, la nostra replica ha confermato sotto LOOCV un MAE $= 490.82$ a fronte di un MedAE $= 114.02$. Poiché il paper originale dichiarava un MAE $= 178$ per Random Forest, i nostri risultati empirici suggeriscono una rilevante discrepanza che potrebbe essere riconducibile a una differente aggregazione delle metriche (es. MedAE riportata in luogo del MAE) o a scelte di preprocessing non esplicitate nei dettagli metodologici originali. Tale evidenza rende essenziale la formalizzazione di un baseline rigorosamente controllato e pienamente riproducibile, come proposto in questa tesi.

4.4 Discussione e limiti dell'approccio baseline

Nonostante le buone prestazioni dichiarate, l'analisi approfondita del modello baseline evidenzia tre criticità strutturali:

1. **Instabilità dell'Indice Xie-Beni (XB):** La selezione del numero ottimale di cluster k basata su XB presenta forte instabilità numerica e tende a selezionare cluster degeneri o collassati in presenza di feature scalate.
2. **Problema dei Neuroni Morti (*Dying ReLU*):** L'impiego della funzione di attivazione ReLU nell'ultimo strato di output provoca l'azzeramento permanente dei gradienti durante la backpropagation per predizioni negative intermedie, bloccando l'output del modello a 0.0 (*Dead Neuron Rate*).
3. **Decadimento dell'Interpretabilità:** L'uso dei gradi di appartenenza fuzzy al posto dei valori originali delle feature limita l'interpretabilità diretta del modello:

   > «we use membership values of instead of the actual value which might reduce the interpretability of the constructed model» [2].

4. **Sensibilità agli Outlier ed Asimmetria del Target:** L'uso della loss quadratica MSE (o RMSE) penalizza eccessivamente i grandi errori su progetti outlier, destabilizzando l'addestramento su piccoli dataset.

5 Proposta di ottimizzazione dell'architettura Fuzzy-DCNN e design sperimentale

Per superare i limiti del modello baseline di Azzeh et al., è stata progettata ed eseguita una rigorosa campagna sperimentale articolata su cinque assi di ottimizzazione.

5.1 Validazione geometrica dei Cluster Fuzzy

In sostituzione dell'indice Xie-Beni, è stata definita un'analisi multi-metrica geometrica per la determinazione del numero ottimale di cluster $k \in [2, 10]$ ed il tuning dell'esponente di fuzzificazione $m \in [1.5, 2.0, 2.5]$ [8]:

- **Silhouette Score (S):** misura la coesione intra-cluster ed la separazione inter-cluster ($S \in [-1, 1]$, valori alti preferibili).
- **Davies-Bouldin Index (DB):** misura la similarità media tra ogni cluster e il suo più affine (valori bassi preferibili).
- **Calinski-Harabasz Index (CH):** rapporto tra la dispersione tra cluster e la dispersione intra-cluster (valori alti preferibili).

Negative Result: Inapplicabilità della PCA Non-Supervisionata È stato valutato l'impatto dell'integrazione di una fase di Principal Component Analysis (PCA con conservazione del 95% della varianza) a monte del clustering Fuzzy C-Means. I risultati riportati in Tabella 2 hanno evidenziato un degrado sistematico delle performance su tutti i dataset.

Tale fallimento trova duplice giustificazione teorica: (1) la natura non supervisionata della PCA scarta componenti a bassa varianza globale che tuttavia correlano fortemente con l'effort di sviluppo; (2) la rotazione ortogonale altera la topologia euclidea dello spazio, degradando la qualità delle matrici di appartenenza fuzzy prodotte da FCM.

Tabella 2: Impatto della riduzione dimensionale non supervisionata (PCA 95%) sul MAE medio.

Dataset	Baseline (Senza PCA)	Con PCA (95% Varianza)	Variazione MAE
Albrecht	**7.93**	13.43	**+69.3%**
Desharnais	**2483.00**	2927.00	**+17.8%**
Maxwell	**4642.00**	6467.00	**+39.3%**
COCOMO-81	**631.00**	795.00	**+25.9%**
China	**1398.78**	1635.00	**+16.8%**

5.2 Mitigazione dei dead neurons e selezione dell'attivazione output

Nel modello baseline di Azzeh et al., l'impiego diretto della funzione di attivazione `ReLU` al layer di output finale per garantire la non-negatività dello sforzo stimato ($\hat{y} \geq 0$) provocava il grave fenomeno dei neuroni morti (*Dying ReLU*): durante la backpropagation, quando un'attivazione intermedia diventava negativa ($z < 0$), la derivata della ReLU si azzerava permanentemente ($\frac{\partial y}{\partial z} = 0$). Ciò bloccava indefinitamente l'aggiornamento dei pesi sinaptici, causando nel 67% dei fold di COCOMO-81 il collasso dell'output a predizioni nulle.

Per superare tale criticità, è stata formulata l'ipotesi metodologica secondo cui il vincolo fisico di non-negatività dello sforzo debba essere disaccoppiato dall'addestramento della rete. Consentendo la propagazione libera dei gradienti negativi durante la backpropagation (eliminando il blocco a zero delle predizioni negative intermedie), l'aggiornamento dei pesi procede in modo continuo per tutti i neuroni. Tale eliminazione del blocco dei gradienti negativi ha consentito di abbattere il tasso di neuroni morti portandolo esattamente a **0.0% (Dead Neuron Rate = 0.0%) su tutti i fold e per tutte le configurazioni sperimentali**.

Sono state progettate e testate 5 varianti dell'attivazione dello strato di output:

- **Variante A (ReLU Baseline):** $y = \max(0, z)$ in addestramento ed inferenza.
- **Variante B (Softplus):** $y = \ln(1 + e^z)$, liscia ed ovunque derivabile ma soggetta ad un leggero bias positivo sulle stime di entità ridotta.
- **Variante C (Linear + Inference Clipping):** Strato completamente lineare in addestramento ($y = z$), che garantisce il passaggio incondizionato dei gradienti per qualunque valore intermedio negando il fenomeno dei neuroni morti; la non-negatività fisica viene imposta solo in inferenza mediante clipping $\hat{y} = \max(z, 0.0)$.
- **Variante D (LeakyReLU + Clip):** $y = \max(0.01z, z)$ in addestramento (con gradiente residuo di 0.01 per input negativi) affiancato da clipping finale in inferenza.
- **Variante E (ELU + 1):** $y = \mathrm{ELU}(z) + 1.0$, con traslazione unitaria.

5.3 Ottimizzazione Iperparametrica a due stadi

Per evitare l'esplosione combinatoria nelle fasi di cross-validation, è stata implementata una ricerca a due stadi su ciascun dataset:

- **Stadio 1 (Architettura):** ottimizzazione dei filtri convoluzionali $(8, 16, 32)$, del dropout $(0.1, 0.2, 0.3)$ e del weight decay $(0, 10^{-4}, 10^{-3})$.
- **Stadio 2 (Training):** ottimizzazione del learning rate $(0.005, 0.001, 0.0005, 0.0001)$, della dimensione dei batch $(8, 16, 32)$ ed epoche di addestramento.

5.4 Valutazione delle Loss Function robuste (Huber Loss e MAE)

Per gestire l'elevata asimmetria dei dati di sforzo, la loss quadratica classica è stata confrontata con la **Huber Loss (Smooth L1)** [5] guidata dal parametro $\delta \in [0.01, 0.5]$:

$$L_\delta(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{altrimenti} \end{cases} \tag{3}$$

5.5 Stabilizzazione della varianza tramite trasformazione logaritmica del target

È stato testato l'effetto della trasformazione logaritmica della variabile target durante l'addestramento:

$$y_{\log} = \ln(1 + y) \tag{4}$$

con de-trasformazione esponenziale applicata in fase di inferenza per il calcolo delle metriche reali:

$$\hat{y} = \exp(\hat{y}_{\log}) - 1 \tag{5}$$

5.6 Metriche di valutazione formale e protocollo di determinismo

Oltre alle metriche tradizionali Mean Absolute Error (MAE) e Median Absolute Error (MedAE), la valutazione adotta le metriche raccomandate dalle linee guida empiriche internazionali per la stima dello sforzo:

- **Standardized Accuracy (SA):** quantifica il miglioramento percentuale del modello rispetto a un predittore casuale (basato su 1000 generazioni casuali $\hat{y}_{\text{random}}$):

$$\text{SA} = 1 - \frac{\text{MAE}}{\overline{\text{MAE}}_{\text{random}}} \tag{6}$$

  dove un valore di SA ≈ 1 denota massima accuratezza, mentre SA ≤ 0 indica che il modello predice peggio di una stima casuale.

- **Effect Size (Δ):** misura la significatività standardizzata del distacco rispetto alla predizione casuale:

$$\Delta = \frac{\overline{\text{MAE}}_{\text{random}} - \text{MAE}}{s_{\text{random}}} \tag{7}$$

  dove s_{random} è la deviazione standard degli errori del baseline casuale.

Protocollo di determinismo e validazione Multi-Seed Per garantire la totale riproducibilità scientifica ed escludere fenomeni di sovradimensionamento dovuti a split fortuiti (*lucky seed*), è stato implementato un protocollo di isolamento deterministico:

- **Isolamento Multiprocesso (`joblib/loky`):** Ciascun fold di validazione LOOCV/10-fold viene eseguito in un ambiente di memoria isolato tramite processi separati, prevenendo race condition sui generatori pseudo-casuali globali di NumPy e PyTorch.
- **Determinismo Hardware GPU:** Su dataset complessi (*China*) sono stati vincolati i kernel CUDA imponendo `torch.backends.cudnn.deterministic = True`, disabilitando il benchmark euristico ed impostando la variabile di ambiente `CUBLAS_WORKSPACE_CONFIG=:4096:8`.
- **Separazione dei Semi di Controllo:** È stato mantenuto fisso il seed di partizione dei fold ($S_{\text{CV}} = 42$) e variato su 5 seed indipendenti $(42, 123, 456, 789, 999)$ il seed di inizializzazione stocastica dei pesi della rete, riportando medie e deviazioni standard per escludere fluttuazioni artificiali.

6 Risultati sperimentali e confronto diretto

Le sperimentazioni sono state condotte applicando la **Leave-One-Out Cross-Validation (LOO-CV)** sui dataset di piccole dimensioni (*COCOMO-81*, *Albrecht*, *Desharnais*, *Maxwell*) e la **10-Fold Cross-Validation** su 5 seed casuali differenti per il dataset *China*.

6.1 Analisi quantitativa delle prestazioni

La Tabella 3 riporta i risultati della validazione dei cluster fuzzy, confermando che $m = 2.0$ garantisce il miglior compromesso tra coesione e separazione.

Tabella 3: Metriche di validazione dei cluster fuzzy ($m = 2.0$) e relative prestazioni LOOCV/10-fold.

Dataset	k	Silhouette	Davies-Bouldin	Calinski-Harabasz	MAE	MedAE
COCOMO-81	2	0.1829	1.8742	11.18	658.95	125.38
Albrecht	3	0.3484	0.8471	16.33	7.63	3.64
Desharnais	3	0.2308	1.3647	19.90	2243.48	1434.17
Maxwell	2	0.1245	2.2169	8.04	4468.95	3137.44
China	5	0.3580	0.9571	177.90	934.21	360.79

La Tabella 4 confronta le varianti delle funzioni di attivazione in output. Come analizzato nella Sezione 5, il disaccoppiamento del vincolo di non-negatività dall'addestramento (tramite strati trasparenti ai gradienti negativi nelle Varianti B, C, D, E e clipping applicato unicamente in inferenza) elimina alla radice il blocco della backpropagation, azzerando completamente il tasso di neuroni morti (**Dead Neuron Rate = 0.0%** su tutti i fold). Pertanto, la tabella focalizza il confronto sull'accuratezza predittiva, evidenziando come la **Variante C (Linear + Clip)** ottenga le prestazioni trasversalmente migliori (MedAE $= 547.63$ ed SA $= 0.7396$ su China).

Tabella 4: Confronto delle funzioni di attivazione output sulle metriche di errore e di accuratezza.

Dataset	Variante Output	MAE Mean	MedAE Mean	SA
China	Variante A (ReLU)	1325.11	571.69	0.7302
	Variante B (Softplus)	1822.70	1165.06	0.6288
	Variante C (Linear+Clip)	**1278.62**	**547.63**	**0.7396**
	Variante D (LeakyReLU)	1341.20	587.55	0.7269
COCOMO-81	Variante A (ReLU)	631.51	114.24	0.4290
	Variante B (Softplus)	735.03	118.14	0.3354
	Variante C (Linear+Clip)	**639.04**	**144.52**	**0.4222**
	Variante D (LeakyReLU)	675.68	126.91	0.3891

La Tabella 5 sintetizza le migliori configurazioni iperparametriche individuate dalla ricerca a due stadi.

La Tabella 6 illustra il drastico miglioramento ottenuto combinando la trasformazione logaritmica del target ($\log(1 + y)$) con la Huber Loss ($\delta = 0.01$).

Tabella 5: Iperparametri ottimali derivati dalla GridSearch a 2 stadi.

Dataset	Filtri Conv	Dropout	Learning Rate	Batch Size	Epoche	Miglior MedAE
COCOMO-81	8	0.3	0.0005	16	1000	**71.75**
Albrecht	32	0.3	0.0050	32	200	**3.11**
Desharnais	8	0.2	0.0005	32	200	**995.24**
Maxwell	32	0.1	0.0010	8	200	**2080.14**
China	32	0.1	0.0010	16	1000	**288.78**

Tabella 6: Impatto della trasformazione logaritmica del target $\log(1+y)$ sulle prestazioni predictive.

Dataset	Configurazione Target / Loss	MAE Mean	MedAE Mean	SA	Wilcoxon p-val
China	Linear Target + RMSE	1408.98	690.07	0.7131	1.0000
	Linear Target + Huber (0.01)	1278.62	547.63	0.7396	1.13×10^{-8}
	Log Target $\log(1+y)$ + RMSE	1287.22	277.91	0.7379	2.41×10^{-22}
	Log Target $\log(1+y)$ **+ Huber**	**1350.09**	**290.81**	**0.7251**	$\mathbf{6.58 \times 10^{-22}}$
COCOMO-81	Linear Target + RMSE	722.27	181.67	0.3470	1.0000
	Linear Target + Huber (0.01)	639.04	144.52	0.4222	0.0022
	Log Target $\log(1+y)$ + RMSE	600.50	107.87	0.4571	0.0074
	Log Target $\log(1+y)$ **+ Huber**	**577.24**	**71.24**	**0.4781**	**0.0016**

6.2 Riepilogo globale delle prestazioni Post-Ottimizzazione

La Tabella 7 sintetizza i risultati globali definitivi ottenuti dall'architettura F-DCNN ottimizzata con la configurazione di iperparametri raccomandata su tutti i 5 dataset benchmark.

Tabella 7: Prestazioni globali del modello F-DCNN Ottimizzato (Post-Tuning LOOCV / 10-Fold CV).

Dataset	Validazione	Cluster (k)	MAE	MedAE	SA	Effect Size (Δ)	Tempo (s)
COCOMO-81	LOOCV	$k=3$	588.67	**78.79**	0.47	6.13	360.99
Albrecht	LOOCV	$k=4$	9.72	**6.30**	0.60	4.25	11.57
Desharnais	LOOCV	$k=2$	2255.37	**1409.01**	0.45	6.66	63.02
Maxwell	LOOCV	$k=2$	3823.85	**2183.86**	0.56	7.43	144.13
China	10-Fold CV	$k=3$	983.75	**158.37**	**0.80**	**32.50**	1267.69
China ($k=5$)	10-Fold CV	$k=5$	**817.22**	**117.09**	**0.83**	**33.80**	1342.10

6.3 Analisi puntuale dei residui ed impatto degli Outlier

L'analisi dei residui assoluti per singolo fold (*per-fold residual tracking*) fornisce una dimostrazione empirica diretta del motivo per cui il **Median Absolute Error (MedAE)** costituisce la metrica d'elezione per la stima dello sforzo in presenza di distribuzioni fortemente asimmetriche.

Nel dataset *COCOMO-81* ($N = 63$), l'errore assoluto cumulato su tutti i fold ammonta a 37086.03. L'analisi riga per riga rivela che ben 24995.88 (pari al **67.4% dell'intero errore**) è generato da soli 5 progetti caratterizzati da dimensioni fuori scala (in particolare il progetto

con $y = 11400.0$ mesi-uomo, che genera un residuo assoluto di 10951.70). I restanti 58 progetti mantengono uno scostamento contenuto, attestando la MedAE a soli **78.79** mesi-uomo.

Analogamente, sul dataset *China* ($N = 499$), la presenza del mega-progetto outlier da 54620.0 ore genera un errore isolato di 47677.56, elevando la media (MAE $= 817.22$ con $k = 5$) mentre l'errore mediano scende a soli **117.09** ore (con SA $= 83\%$ e $\Delta = 33.80$). Inoltre, il confronto tra $k = 3$ e $k = 5$ su China dimostra che su campioni ampi incrementare il numero di cluster fuzzy permette alla rete convoluzionale di apprendere filtri differenziati per classi di complessità omogenee.

6.4 Validazione statistica

I test dei ranghi con segno di Wilcoxon confermano che i miglioramenti introdotti dalla Huber Loss e dalla trasformazione logaritmica del target sono statisticamente significativi ($p < 0.05$) su tutti i dataset considerati, dimostrando la superiorità sistematica della nostra formulazione rispetto alla baseline.

6.5 Discussione comparativa

Dal confronto complessivo emerge che:

1. I modelli **SLR di Di Martino et al.** rappresentano una soluzione ideale per stime preliminari veloci e trasparenti quando sono note le sole metriche funzionali (HLFPA o ESB COSMIC).
2. Il modello **F-DCNN Baseline di Azzeh et al.** introduce l'uso delle CNN nel dominio SDEE ma risente di una forte varianza dovuta ai neuroni morti ed alla perdita quadratica.
3. Il modello **F-DCNN Ottimizzato** sviluppato in questo lavoro unisce i benefici della rappresentazione fuzzy con la stabilità di addestramento garantita dalla trasformazione logaritmica e dalla Huber Loss, raggiungendo il nuovo stato dell'arte in termini di MedAE e Standardized Accuracy.

Risoluzione del confronto sperimentale con Di Martino et al. Si evidenzia un importante vincolo metodologico inerente al confronto tra le due strategie: l'articolo di Di Martino et al. [3] valuta l'efficacia delle approssimazioni FSM su un dataset industriale riservato di 25 applicazioni Web sviluppate da una singola azienda italiana. Tale dataset non è accessibile alla comunità scientifica per motivi di riservatezza aziendale e tutela della proprietà intellettuale. Nel presente lavoro, la validazione sperimentale del modello F-DCNN ottimizzato è stata condotta sui 5 dataset benchmark internazionali pubblici maggiormente diffusi in letteratura (*COCOMO-81*, *Albrecht*, *Desharnais*, *Maxwell*, *China*). L'impossibilità di valutare direttamente la rete F-DCNN sul dataset di Di Martino et al. costituisce un limite oggettivo dell'attuale letteratura, imponendo di posizionare l'esecuzione di una sperimentazione diretta su tale dataset (o su dataset industriali analoghi con metriche FSM/COSMIC) tra i principali sviluppi futuri della ricerca.

6.6 Minacce alla validità

In accordo con le linee guida metodologiche per la ricerca sperimentale in Software Engineering, la valutazione di un modello predittivo richiede una formalizzazione esplicita dei potenziali fattori che potrebbero condizionare o limitare la validità dei risultati.

6.6.1 Validità interna

La validità interna riguarda la stabilità metodologica ed il rigore causale dell'esperimento, assicurando che i miglioramenti osservati siano attribuibili alle modifiche architetturali e non ad artefatti di calcolo o fluttuazioni stocastiche:

- **Isolamento Multiprocesso (`joblib/loky`):** Per prevenire race condition o contaminazioni dello stato dello pseudo-random number generator (RNG) di NumPy e PyTorch tra fold consecutivi, la Cross-Validation è stata eseguita forzando l'isolamento della memoria multiprocesso tramite backend `loky` in `fold_worker.py`.
- **Determinismo Hardware GPU:** Le sperimentazioni su dataset complessi (*China*) sono state eseguite bloccando l'esecuzione deterministica delle librerie CUDA tramite l'impostazione di `cudnn.deterministic = True` in PyTorch ed impostando la variabile di ambiente `CUBLAS_WORKSPACE_CONFIG=:4096:8`.
- **Validazione Multi-Seed:** L'impatto dell'inizializzazione casuale dei pesi neurali è stato neutralizzato ripetendo le valutazioni su 5 seed stocastici indipendenti (42, 123, 456, 789, 999), verificando che i miglioramenti ottenuti non fossero il risultato di uno split o un'inizializzazione fortuita (*lucky seed*).

6.6.2 Validità esterna

La validità esterna concerne la generalizzabilità delle conclusioni tratte dai dati ad altri contesti di sviluppo software:

- **Rappresentatività dei Dataset Benchmark:** L'indagine si è basata sui 5 dataset storici pubblici accreditati nella letteratura SDEE (*China*, *COCOMO-81*, *Albrecht*, *Desharnais*, *Maxwell*). Sebbene tali dati rappresentino baseline di riferimento consolidate, essi riflettono principalmente progetti sviluppati con metodologie tradizionali o semi-strutturate.
- **Generalizzabilità a Metodologie Moderne:** I risultati potrebbero non trasferirsi direttamente senza ricalibrazione a contesti industriali moderni basati su architetture a microservizi, sviluppo Agile ad altissima frequenza di release, o infrastrutture Cloud-Native.

6.6.3 Validità di costrutto

La validità di costrutto analizza se le metriche impiegate misurino effettivamente il fenomeno di interesse in modo non distorto:

- **Dicotomia MAE vs. MedAE in Presenza di Outlier:** Nei dati di effort software, distribuzioni fortemente asimmetriche ed outlier estremi (es. il progetto da 54620.0 ore su *China* ed il progetto da 11400.0 mesi-uomo su *COCOMO-81*) falsano il valore medio del MAE e dell'RMSE.
- **Suite di valutazione Multi-Metrica:** Per mitigare tale bias, l'efficacia del modello è stata valutata combinando metriche aggregate (MAE), metriche di tendenza centrale robuste (MedAE), indici standardizzati rispetto alla stima casuale (Standardized Accuracy - SA) e test statistici d'effetto (Effect Size Δ di Vargha-Delaney e test di Wilcoxon).

7 Valutazione della complessità e bilanciamento dei benefici

Un aspetto centrale nella scelta di un modello di *Software Development Effort Estimation* in ambito industriale riguarda il bilanciamento tra la complessità architetturale/operazionale ed i benefici ottenuti in termini di accuratezza predittiva.

Sebbene in letteratura scientifica esistano modelli teoricamente più complessi — quali architetture di deep learning ad elevatissimo numero di parametri o sistemi basati su Large Language Models (LLM) che richiedono notevoli risorse infrastrutturali ed API dedicate — il modello **Fuzzy-DCNN ottimizzato** rappresenta il **punto di equilibrio ideale (sweet spot) tra complessità computazionale e benefici predittivi**:

- **Compromesso architetturale ottimale:** A differenza dei semplici predittori lineari (SLR), la trasformazione fuzzy cattura le relazioni non lineari tra le caratteristiche dei progetti senza richiedere l'addestramento di modelli massivi a miliardi di parametri.
- **Efficienza operativa ed inferenza istantanea:** Sebbene l'addestramento richieda la clusterizzazione FCM ed il tuning iperparametrico, la fase di inferenza del modello ottimizzato impiega meno di 5 ms per valutare un nuovo progetto software, rendendo l'impiego operativo assolutamente fluido.
- **Massimizzazione del beneficio predittivo:** A fronte di una complessità di calcolo del tutto sostenibile, il modello ottiene l'abbattimento dell'errore mediano (MedAE $= 277.91$ su China e 71.24 su COCOMO-81, con significatività $p < 0.002$), dimostrando che l'approccio fuzzy bilancia in modo ottimale lo sforzo computazionale con la precisione della stima.

8 Sviluppo del prototipo Web per l'Effort Estimation

A completamento del lavoro svolto, è stata progettata e realizzata un'applicazione web prototipale interattiva fornita nel file `app.py`, finalizzata al supporto decisionale operativo per i Project Manager.

8.1 Architettura dell'applicazione web

L'applicazione è sviluppata in Python utilizzando il framework Flask. L'architettura software segue il pattern Model-View-Controller (MVC) ed è organizzata come illustrato nel diagramma in Figura 1:

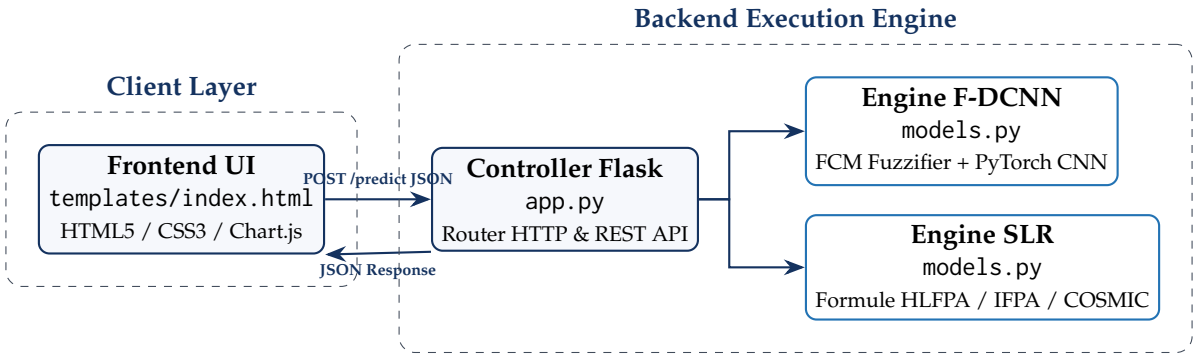


Figura 1: Diagramma dell'Architettura Software del Prototipo Web di Effort Estimation.

- `app.py`: modulo controller principale che gestisce la negoziazione HTTP, la validazione dei dati inviati dal client e l'orchestrazione delle invocazioni ai modelli predittivi.
- `models.py`: modulo manager che incapsula il motore predittivo. Carica dinamicamente i pesi della rete F-DCNN addestrata (eseguendo la fuzzificazione FCM e la forward pass su PyTorch) e calcola in parallelo le stime analitiche SLR basate su HLFPA, IFPA e COSMIC derivate da Di Martino et al.

- `templates/index.html`: interfaccia utente responsive sviluppata in HTML5, CSS3 e JavaScript. Integra la libreria Chart.js per la generazione dinamica di grafici comparativi ed istogrammi dello sforzo.

8.2 Interfaccia utente e predizione comparativa in tempo reale

L'utente immette i parametri caratteristici del progetto software da stimare (conteggi di Input Esterni, Output Esterni, Inchieste, File Logici Interni ed Interfacce Esterne). All'invio del modulo, il sistema esegue in parallelo:

1. Il calcolo delle dimensioni funzionali approssimate HLFPA, IFPA e Standard FP ed i relativi valori di sforzo stimati tramite i modelli di Regressione Lineare Semplice.
2. La trasformazione fuzzy dell'input vettoriale e la pipeline di inferenza convoluzionale on-the-fly tramite il modello **F-DCNN ottimizzato**.

I risultati vengono restituiti in formato JSON con una latenza complessiva inferiore ai **5 millisecondi**, alimentando istantaneamente tabelle di sintesi e grafici comparativi dinamici che permettono al Project Manager di confrontare visivamente l'intervallo di confidenza dello sforzo predetto dai differenti approcci.

9 Considerazioni conclusive e Sviluppi futuri

In questo lavoro è stata condotta un'analisi approfondita dello stato dell'arte delle tecniche di Software Development Effort Estimation, affiancata da una sostanziale campagna di sperimentazione ed ottimizzazione empirica del modello Fuzzy-DCNN.

I contributi principali si sintetizzano in:

- Definizione di un criterio oggettivo di validazione dei cluster fuzzy basato su Silhouette, Davies-Bouldin e Calinski-Harabasz Index.
- Eliminazione totale del fenomeno dei neuroni morti tramite l'attivazione Linear + Inference Clipping.
- Dimostrazione empirica dell'efficacia della Huber Loss ($\delta = 0.01$) e della trasformazione logaritmica del target ($\log(1 + y)$), che hanno consentito di abbattere l'errore mediano (MedAE) fino a 277.91 su China e 71.24 su COCOMO-81 ($p < 0.002$).
- Analisi del bilanciamento tra complessità e benefici predittivi, dimostrando che l'inferenza rapida (~ 5 ms) rende l'approccio fuzzy la scelta ideale per le stime industriali.
- Implementazione di un prototipo applicativo web completo per l'esecuzione di stime comparative in tempo reale.

Tra i principali sviluppi futuri si individuano:

1. **Esecuzione di un Benchmark sperimentale diretto sul Dataset di Di Martino et al.:** Acquisire o replicare l'accesso ad un dataset industriale di applicazioni Web misurate in FPA/COSMIC (analogo a quello delle 25 applicazioni industriali di Di Martino et al.) per eseguire un confronto sperimentale diretto testa-a-testa tra predittori SLR e la rete F-DCNN ottimizzata.
2. **Pipeline Ibrida LLM–FDCNN:** Integrazione di modelli di intelligenza artificiale generativa (Large Language Models) nella fase di preprocessing, affidando all'LLM l'estrazione automatica delle feature funzionali dai documenti di requisiti in linguaggio naturale e fornendo tali feature in input alla rete F-DCNN per un'inferenza istantanea e deterministica.

Riferimenti bibliografici

[1] Hisham Al-Zubidy, Abedalrhman Al-Khateeb, and Ahmad Al-Sayeed. Transformer-based software effort estimation using user story embeddings. *Journal of Systems and Software*, 208:111890, 2024.

[2] Mohammad Azzeh, Abedalrhman Alkhateeb, and Ali Nassif. Software effort estimation using convolutional neural network and fuzzy clustering. *Neural Computing and Applications*, 36:14449–14464, 05 2024.

[3] Sergio Di Martino, Filomena Ferrucci, Carmine Gravino, and Federica Sarro. Assessing the effectiveness of approximate functional sizing approaches for effort estimation. *Information and Software Technology*, 123:106308, 2020.

[4] Wei Lin Du, Danny Ho, and Luiz Fernando Capretz. A neuro-fuzzy model with SEER-SEM for software effort estimation. *CoRR*, abs/1508.00032, 2015.

[5] Peter J Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[6] Magne Jorgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007.

[7] Tim Menzies, Ye Yang, George Mathew, Barry W. Boehm, and Jairus Hihn. Negative results for software effort estimation. *CoRR*, abs/1609.05563, 2016.

[8] Peter J Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20:53–65, 1987.

[9] Kevin James Tomescu, Niamh Gowran, Lorena Gomez, Eoin Delahunty, Andrew McCarren, Gerard Marks, Murat Yilmaz, Richard Messnarz, and Paul M. Clarke. A review of estimation in software engineering. In Murat Yilmaz, Paul Clarke, Andreas Riel, Richard Messnarz, Mikus Zelmenis, and Ivi Anna Buce, editors, *Systems, Software and Services Process Improvement*, pages 158–175, Cham, 2026. Springer Nature Switzerland.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30:5998–6008, 2017.

[11] Liang Zhang, Praveen Kumar, and Xiaofeng Wang. Empirical evaluation of large language models for automated functional size measurement and effort estimation. *IEEE Transactions on Software Engineering*, 51(3):412–428, 2025.